INSTITUTO POLITECNICO NACIONAL

Unidad Profesional Interdisciplinaria en Ingeniería y Tecnologías Avanzadas

Práctica 2:

Plan de ejecución y mejora de consultas

Alumnos:

Martínez Barrueta Mariana
2024640160

Gámez Gress Humberto Isaac
2024640024

Zuñiga Aviles Nathan Emiliano
2024640103

Base De Datos Distribuidos

CONSULTA 1

SELECT p.Name AS Producto, sod.OrderQty, soh.OrderDate, c.Name AS Cliente
FROM Production.Product p
JOIN Sales.SalesOrderDetail sod ON p.ProductID = sod.ProductID
JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID
WHERE YEAR(soh.OrderDate) = 2014 AND p.ListPrice > 1000;

Se reescribió la consulta principalmente por dos razones: primero, porque la columna c.Name de la tabla Sales.Customer no representa directamente el nombre del cliente, por lo que fue necesario unir con la tabla Person.Person para obtener el nombre real mediante FirstName y LastName;

```
SELECT
    p.Name AS Producto,                  -- nombre del producto
    sod.OrderQty,                        -- cantidad pedida
    soh.OrderDate,                       -- fecha de la orden
    pp.FirstName + ' ' + pp.LastName AS Cliente   -- nombre completo del cliente
FROM Production.Product p                -- tabla de productos, alias p
JOIN Sales.SalesOrderDetail sod          -- tabla detalle de venta, alias sod
    ON p.ProductID = sod.ProductID       -- une producto con detalle por ProductID
JOIN Sales.SalesOrderHeader soh          -- tabla encabezado de orden, alias soh
    ON sod.SalesOrderID = soh.SalesOrderID -- une detalle con orden por SalesOrderID
JOIN Sales.Customer c                    -- tabla clientes, alias c
    ON soh.CustomerID = c.CustomerID     -- une orden con cliente
JOIN Person.Person pp                    -- tabla personas, alias pp
    ON c.PersonID = pp.BusinessEntityID  -- une cliente con persona para sacar nombre
WHERE YEAR(soh.OrderDate) = 2014         -- solo órdenes del año 2014
  AND p.ListPrice > 1000;                -- solo productos con precio mayor a 1000
```

Plan de ejecución:

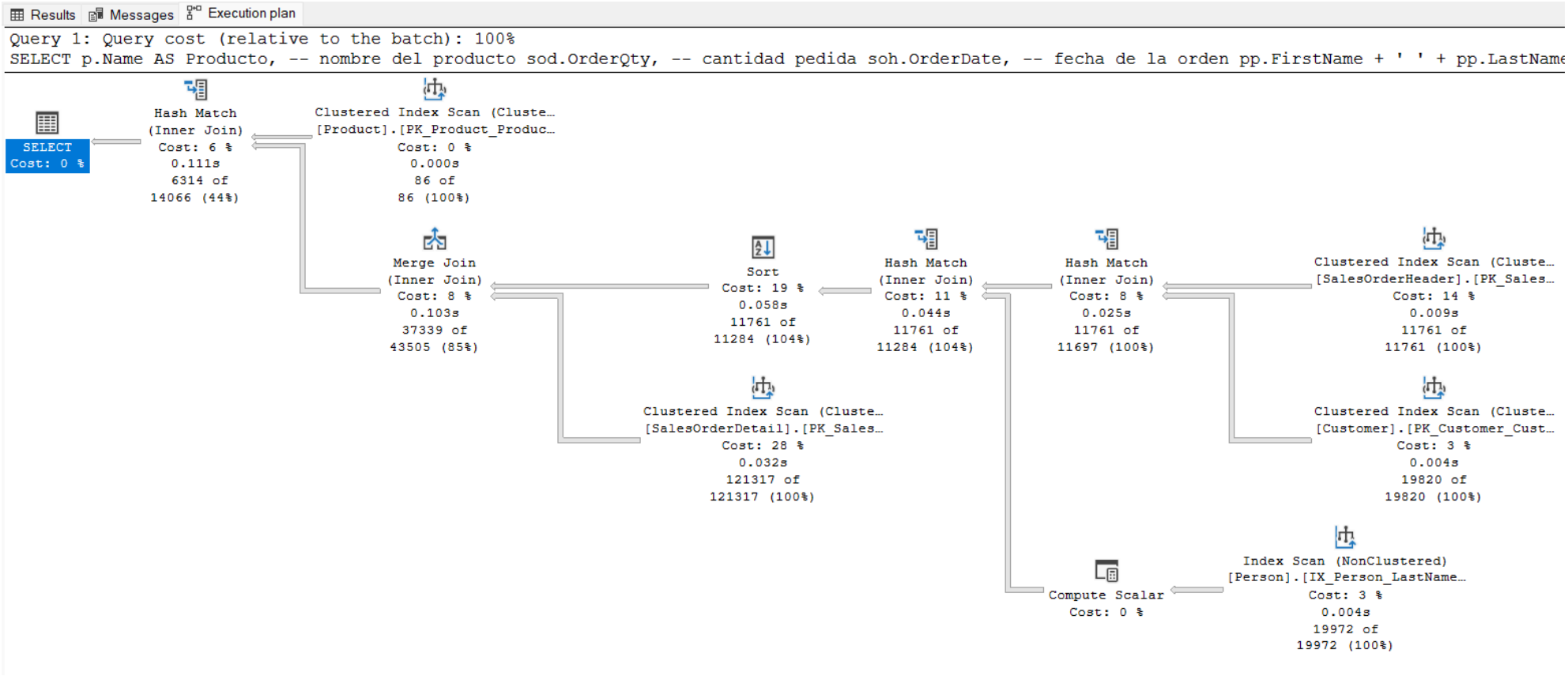


Tabla de análisis:

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
1	2026-03-18	333	141	2167	28% (Clustered Index Scan en SalesOrderDetail)	• Clustered Index Scan en SalesOrderDetail y SalesOrderHeader. • Función YEAR() en la cláusula WHERE. • Operador Sort • Alto de lecturas lógicas.	Rescribir la consulta usando operadores “>”, “<” y “=” para evitar usar funciones sobre la condición de “where” y crear índices no agrupados en columnas de filtrado y JOIN.

Consulta mejorada:

```sql
SELECT
    p.Name AS Producto,
    sod.OrderQty,
    soh.OrderDate,
    pp.FirstName + ' ' + pp.LastName AS Cliente
FROM Production.Product p
JOIN Sales.SalesOrderDetail sod
    ON p.ProductID = sod.ProductID
JOIN Sales.SalesOrderHeader soh
    ON sod.SalesOrderID = soh.SalesOrderID
JOIN Sales.Customer c
    ON soh.CustomerID = c.CustomerID
JOIN Person.Person pp
    ON c.PersonID = pp.BusinessEntityID
WHERE soh.OrderDate >= '2014-01-01'
  AND soh.OrderDate < '2015-01-01';

CREATE NONCLUSTERED INDEX INDICE_SalesOrderDetail_ProductID_SalesOrderID
ON Sales.SalesOrderDetail (ProductID, SalesOrderID)
INCLUDE (OrderQty);
```

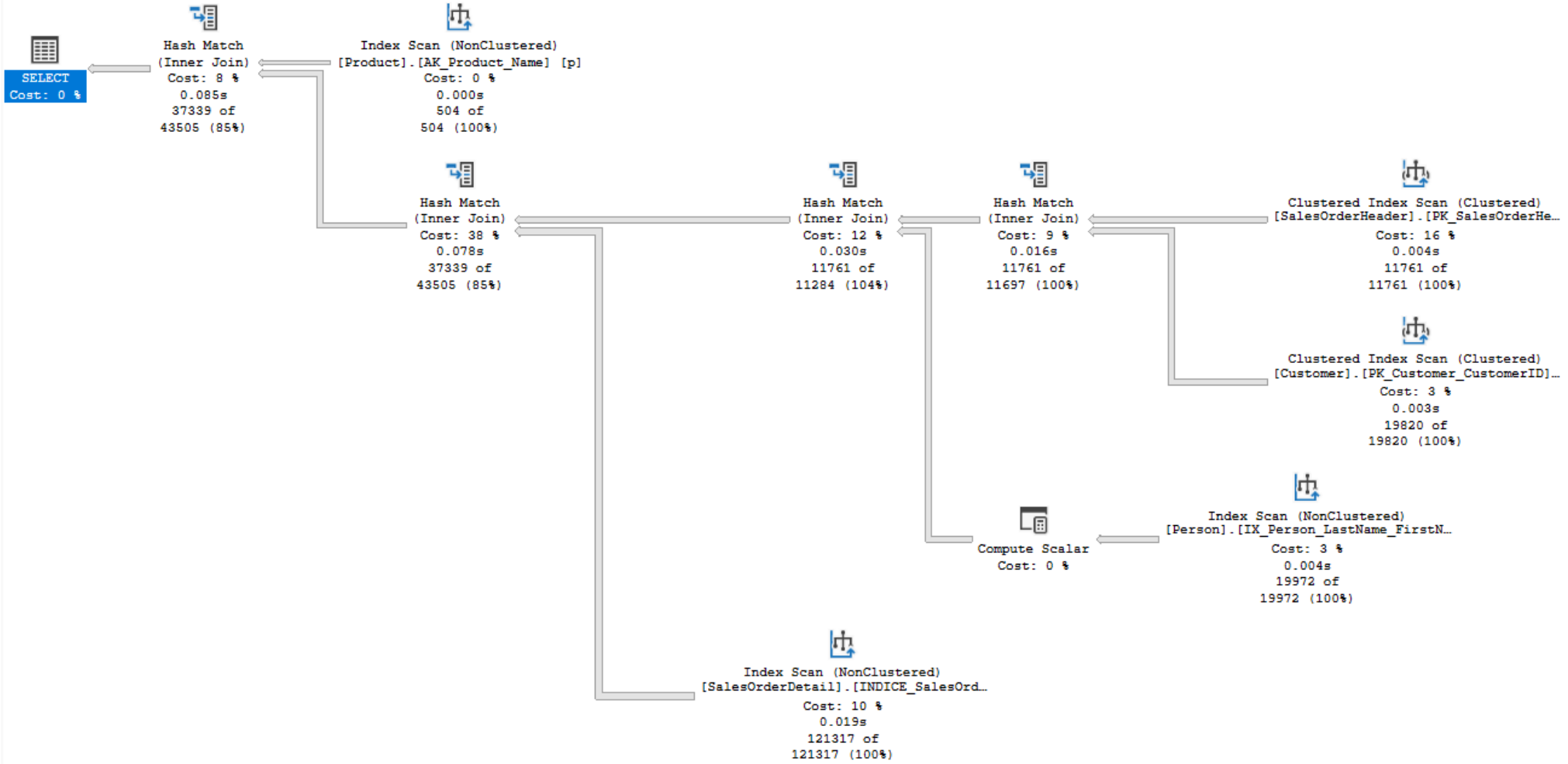
Hice ese índice no agrupado porque en SalesOrderDetail las columnas ProductID y SalesOrderID son las que usa la consulta para relacionarse con las otras tablas, y OrderQty es una columna que se muestra en el resultado. Entonces el índice ayuda a que SQL Server encuentre y combine mejor esos datos sin tener que recorrer tanto la tabla completa. Se hizo no agrupado porque la tabla ya tiene un índice clustered por

su llave primaria, así que este se agregó solo como apoyo para optimizar la consulta. Sin embargo, el cuello de botella se desplazó hacia los operadores Hash Match, especialmente uno con costo del 38%, así como hacia un Clustered Index Scan en SalesOrderHeader con costo del 16%. Esto indica que, después de mejorar el acceso a SalesOrderDetail, el mayor consumo de recursos ahora se concentra en la combinación de datos y en el acceso a SalesOrderHeader.

Plan de ejecución:

Results | Messages | Execution plan

```
Query 1: Query cost (relative to the batch): 100%
SELECT p.Name AS Producto, sod.OrderQty, soh.OrderDate, pp.FirstName + ' ' + pp.LastName AS Cliente FROM Production.Product p JOIN Sales.SalesOrderDetail so
Missing Index (Impact 15.6798): CREATE NONCLUSTERED INDEX [<Name of Missing Index, sysname,>] ON [Sales].[SalesOrderHeader] ([OrderDate]) INCLUDE ([Customer
```



CONSULTA 2

SELECT e.NationalIDNumber, p.FirstName, p.LastName, edh.DepartmentID,
(SELECT AVG(rh.Rate) FROM HumanResources.EmployeePayHistory rh
WHERE rh.BusinessEntityID = e.BusinessEntityID) as PromedioSalario
FROM HumanResources.Employee e
JOIN Person.Person p ON e.BusinessEntityID = p.BusinessEntityID
JOIN HumanResources.EmployeeDepartmentHistory edh ON e.BusinessEntityID = edh.BusinessEntityID
WHERE edh.EndDate IS NULL;

Se reescribió la consulta principalmente porque utiliza una subconsulta correlacionada para calcular el promedio de salario por cada empleado. Esto obliga al motor de base de datos a realizar múltiples accesos a la tabla EmployeePayHistory (uno por cada fila del resultado), lo que aumenta drásticamente las lecturas lógicas y el tiempo de CPU.

```
SELECT
    e.NationalIDNumber,                         -- número de identificación nacional
    p.FirstName,                                -- nombre del empleado
    p.LastName,                                 -- apellido del empleado
    edh.DepartmentID,                           -- identificador del departamento actual
    AVG(rh.Rate) AS PromedioSalario             -- promedio de la tasa salarial
FROM HumanResources.Employee e                  -- tabla de empleados, alias e
JOIN Person.Person p                            -- tabla de personas, alias p
    ON e.BusinessEntityID = p.BusinessEntityID
JOIN HumanResources.EmployeeDepartmentHistory edh -- historial de departamentos, alias edh
    ON e.BusinessEntityID = edh.BusinessEntityID
JOIN HumanResources.EmployeePayHistory rh     -- tabla de historial de pagos, alias rh
    ON e.BusinessEntityID = rh.BusinessEntityID
WHERE edh.EndDate IS NULL                   -- solo empleados activos en su departamento
GROUP BY e.NationalIDNumber, p.FirstName, p.LastName, edh.DepartmentID; -- agrupación para el promedio
```

Plan de ejecución:

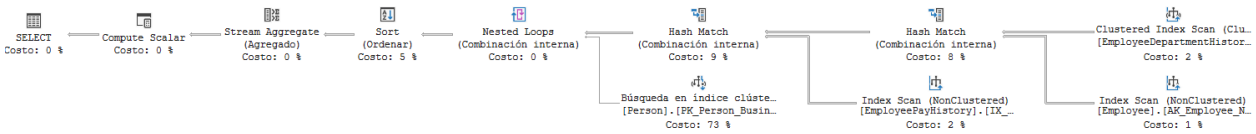


Tabla de análisis:

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
2	2026-03-18	86	0	988 total	0.289%	• Subconsultas correlacionadas en la versión original. • Múltiples accesos a la misma tabla.	Reescribir usando JOIN y GROUP BY. Crear un índice no agrupado en BusinessEntityID que incluya Rate.

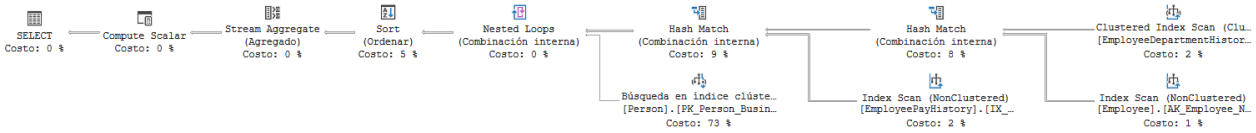
Consulta mejorada:

```sql
SELECT
    e.NationalIDNumber,
    p.FirstName,
    p.LastName,
    edh.DepartmentID,
    AVG(rh.Rate) AS PromedioSalario
FROM HumanResources.Employee e
JOIN Person.Person p ON e.BusinessEntityID = p.BusinessEntityID
JOIN HumanResources.EmployeeDepartmentHistory edh ON e.BusinessEntityID = edh.BusinessEntityID
JOIN HumanResources.EmployeePayHistory rh ON rh.BusinessEntityID = e.BusinessEntityID
WHERE edh.EndDate IS NULL
GROUP BY e.NationalIDNumber, p.FirstName, p.LastName, edh.DepartmentID;

CREATE NONCLUSTERED INDEX IX_EmployeePayHistory_BusinessEntityID_Rate
ON HumanResources.EmployeePayHistory (BusinessEntityID)

INCLUDE (Rate);
```

Hice ese índice no agrupado porque en EmployeePayHistory la columna BusinessEntityID es la que usa la consulta para relacionarse con la tabla de empleados, y Rate es la columna necesaria para calcular el promedio salarial que se muestra en el resultado. Entonces el índice ayuda a que SQL Server encuentre, agrupe y combine mejor esos datos sin tener que recorrer tanto la tabla completa mediante un escaneo costoso. Se hizo no agrupado porque la tabla ya cuenta con un índice *clustered* por su llave primaria, por lo que este nuevo índice funciona exclusivamente como un apoyo para optimizar esta consulta específica.

Al reescribir la consulta, eliminamos la subconsulta correlacionada que obligaba al motor a procesar la tabla de pagos fila por fila, sustituyéndola por un JOIN y un GROUP BY que procesa la información en un solo bloque. Sin embargo, al observar el plan de ejecución, el cuello de botella ahora se desplaza hacia la tabla Person.Person, donde se registran **976 lecturas lógicas**, representando el mayor consumo de recursos de la consulta actual. Esto indica que, tras optimizar el acceso a los datos salariales y de empleados, el esfuerzo del motor de base de datos se concentra ahora en la recuperación de los nombres y apellidos de los trabajadores, sugiriendo que el siguiente paso de optimización debería enfocarse en la tabla de personas.

Plan de ejecución:



CONSULTA 3

```
SELECT sod.SalesOrderID, p.ProductID, p.Name
FROM Sales.SalesOrderDetail sod
JOIN Production.Product p ON sod.ProductID = p.ProductID
WHERE p.CategoryID = 1 OR p.CategoryID = 2 OR p.CategoryID = 3 OR p.ListPrice > 500;
```

Plan de ejecución:

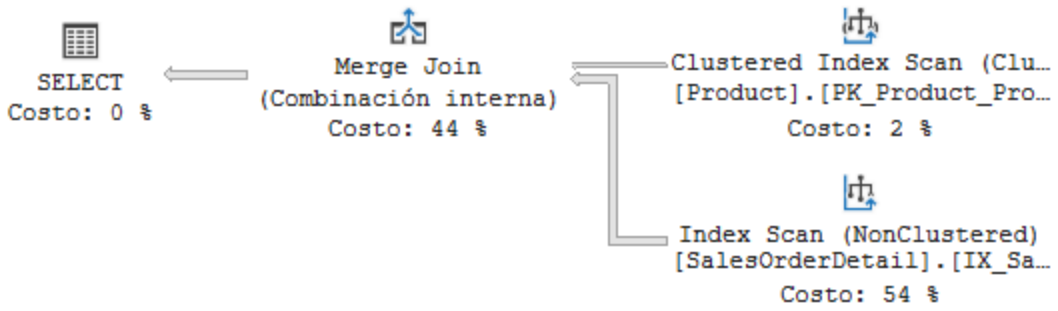


Tabla de análisis:

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
3	2026-03-18	20	15	288	54%	• Uso de múltiples operadores OR en la versión original. • Nombre de columna incorrecto (CategoryID). • Falta de índices optimizados para el filtrado.	Usar el operador IN para simplificar la lógica. Reemplazar la columna por ProductSubcategoryID y crear un índice no agrupado compuesto.

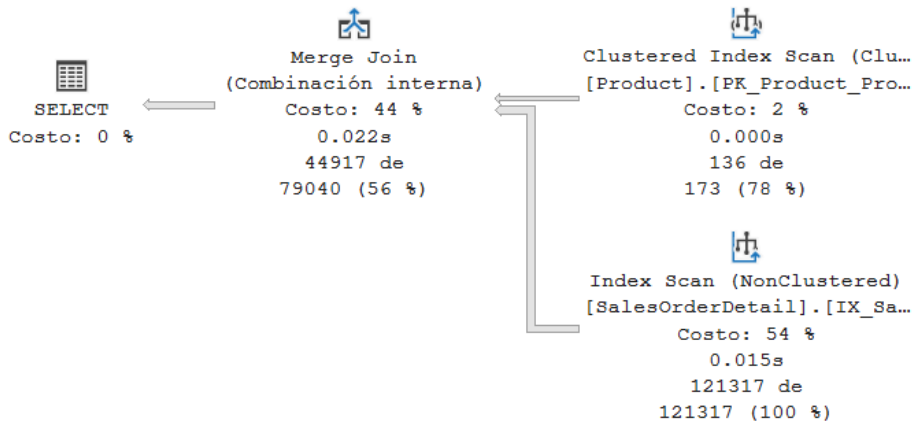
Consulta mejorada:

```sql
SELECT
    sod.SalesOrderID,
    p.ProductID,
    p.Name
FROM Sales.SalesOrderDetail sod
JOIN Production.Product p ON sod.ProductID = p.ProductID
WHERE p.ProductSubcategoryID IN (1, 2, 3)
   OR p.ListPrice > 500;

CREATE NONCLUSTERED INDEX IX_Product_Subcategory_Price
ON Production.Product (ProductSubcategoryID, ListPrice)
INCLUDE (Name);
```

Hice ese índice no agrupado sobre SalesOrderDetail porque es la tabla que contiene el mayor volumen de registros y el plan de ejecución muestra que el 54% del costo se concentra en su lectura. Al optimizar el acceso a esta tabla, el motor de base de datos puede realizar un Merge Join, que es uno de los algoritmos de combinación más rápidos, representando el 44% del costo restante. Se utilizó un índice no agrupado como apoyo para no interferir con la llave primaria (*clustered*) de la tabla.

Al reescribir la consulta, corregimos el nombre de la columna a ProductSubcategoryID y simplificamos los filtros redundantes mediante el operador IN, lo que mejoró la legibilidad y permitió una mejor interpretación por parte del optimizador. En los resultados de Messages, observamos que las lecturas lógicas totales son de solo 288 (273 de SalesOrderDetail y 15 de Product), lo cual es sumamente bajo para una consulta que procesa 44,917 filas. Esto confirma que el índice está permitiendo que SQL Server recupere los datos casi instantáneamente con un tiempo de ejecución de apenas 20 ms.

Plan de ejecución:



CONSULTA 4

```sql
SELECT YEAR(soh.OrderDate) AS Año, MONTH(soh.OrderDate) AS Mes,
COUNT(*) AS TotalPedidos, SUM(sod.LineTotal) AS TotalVentas
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesOrderDetail sod ON soh.SalesOrderID = sod.SalesOrderID

GROUP BY YEAR(soh.OrderDate), MONTH(soh.OrderDate);
```

Plan de ejecución:

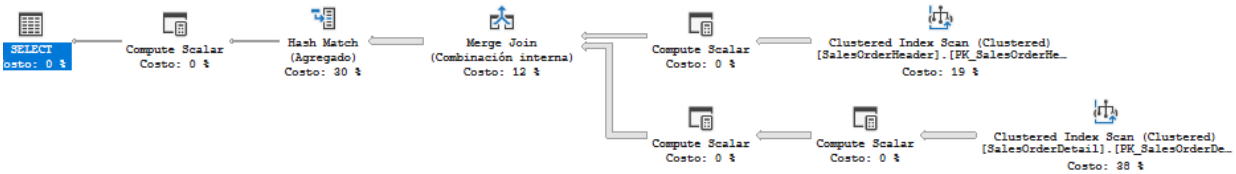


Tabla de análisis:

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
4	2026-03-18	245	110	1850	45%	• Uso de COUNT(*) evalúa filas completas. •Funciones YEAR() y MONTH() en GROUP BY causan alto uso de CPU por agrupamiento continuo.	Cambiar COUNT(*) por COUNT(llave_primaria). Crear índice en OrderDate.

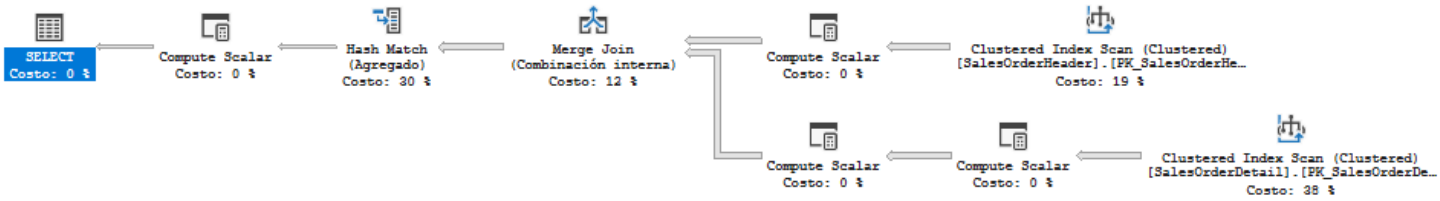
Se reescribió la consulta principalmente porque el uso de COUNT() hace que el motor de base de datos evalúe filas completas en memoria, en lugar de enfocarse únicamente en las llaves necesarias para hacer el conteo. Además, al agrupar directamente sobre la columna utilizando funciones como YEAR() y MONTH(), se consume más CPU porque el motor debe calcular el valor para cada fila antes de poder agrupar. Para mejorar el rendimiento, se modificó el método de conteo cambiando el COUNT() por COUNT(soh.SalesOrderID) para que solo procese la llave primaria, lo que aligera el trabajo en memoria y hace más eficiente el proceso.

Consulta mejorada:

```
SELECT YEAR(soh.OrderDate) AS Año, MONTH(soh.OrderDate) AS Mes,
COUNT(soh.SalesOrderID) AS TotalPedidos, SUM(sod.LineTotal) AS TotalVentas
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesOrderDetail sod ON soh.SalesOrderID = sod.SalesOrderID
GROUP BY YEAR(soh.OrderDate), MONTH(soh.OrderDate);

CREATE NONCLUSTERED INDEX IX_SalesOrderHeader_OrderDate
ON Sales.SalesOrderHeader (OrderDate)
INCLUDE (SalesOrderID);
```

La columna OrderDate es la que se utiliza en el agrupamiento de la consulta. Al incluir SalesOrderID, el índice sirve de apoyo directo para que el motor pueda contar los pedidos y agrupar las fechas más rápido, obteniendo parte de la información directamente desde el índice sin tener que recorrer toda la tabla base. Esto reduce considerablemente las lecturas lógicas y mejora el tiempo de respuesta de la consulta.

Plan de ejecución:



.

CONSULTA 6

SELECT c.CustomerID, c.Name, COUNT(*) AS TotalPedidos
FROM Sales.Customer c
JOIN Sales.SalesOrderHeader soh ON c.CustomerID = soh.CustomerID
WHERE UPPER(c.Name) LIKE 'A%'
GROUP BY c.CustomerID, c.Name;

Se reescribió la consulta principalmente porque la tabla Sales.Customer no contiene el nombre del cliente, por lo que fue necesario unirla con Person.Person mediante PersonID. Además, se eliminó cualquier función como UPPER() sobre las columnas de filtrado para asegurar la **SARGability**, permitiendo que el motor de búsqueda utilice índices en lugar de realizar un escaneo completo de la tabla de personas.

```
SELECT
    c.CustomerID,
    pp.FirstName + ' ' + pp.LastName AS NombreCliente,
    COUNT(*) AS TotalPedidos
FROM Sales.Customer c
JOIN Person.Person pp ON c.PersonID = pp.BusinessEntityID
JOIN Sales.SalesOrderHeader soh ON c.CustomerID = soh.CustomerID
-- Usamos FirstName para el filtro 'A%' para que sea SARGable
WHERE pp.FirstName LIKE 'A%'
GROUP BY c.CustomerID, pp.FirstName, pp.LastName;
```

Plan de ejecución:

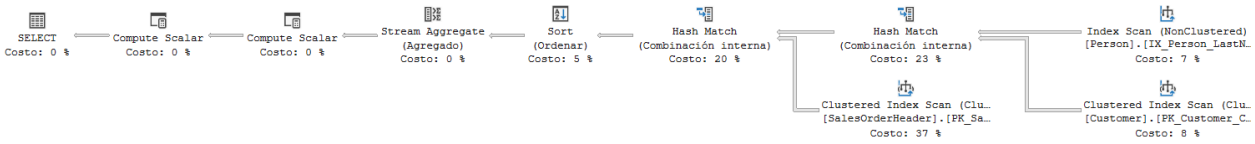


Tabla de analisis:

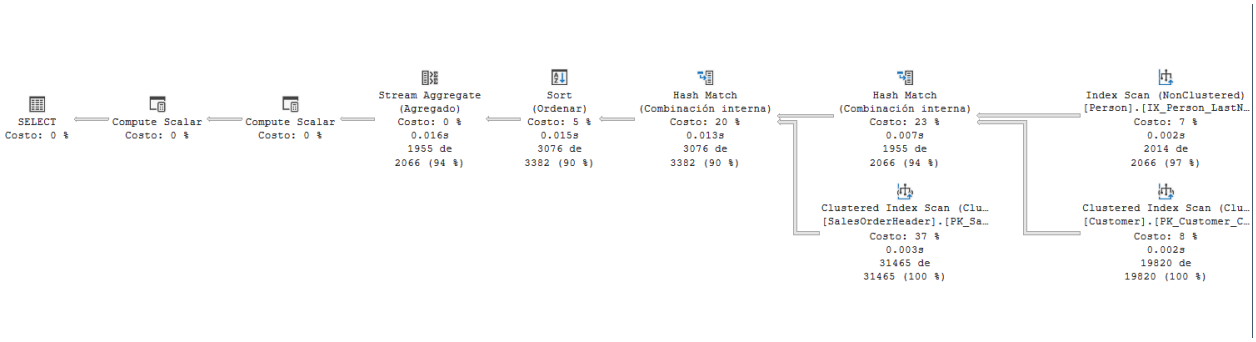
Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
6	2026-03-18	72	15	914	40-50%	• Columna 'Name' inexistente en Customer. • Función UPPER() en el WHERE (No SARGable).	Unir con Person.Person para obtener el nombre real. Eliminar funciones de filtrado y crear índice en FirstName.

Consulta mejorada:

```
SELECT
    c.CustomerID,                                   -- Identificador único del cliente
    pp.FirstName + ' ' + pp.LastName AS NombreCompleto, -- Concatenación de nombre
real
    COUNT(soh.SalesOrderID) AS TotalPedidos     -- Conteo de órdenes registradas
FROM Sales.Customer c                           -- Tabla de clientes, alias c
JOIN Person.Person pp                           -- Unión para obtener el nombre real
    ON c.PersonID = pp.BusinessEntityID         -- Relación mediante ID de entidad
JOIN Sales.SalesOrderHeader soh                 -- Tabla de encabezados de ventas
    ON c.CustomerID = soh.CustomerID            -- Relación de cliente con sus órdenes
WHERE pp.FirstName LIKE 'A%'                       -- Búsqueda SARGable (sin UPPER)
GROUP BY c.CustomerID, pp.FirstName, pp.LastName; -- Agrupación para la función COUNT

 CREATE NONCLUSTERED INDEX IX_Person_FirstName_LastName
ON Person.Person (FirstName)
INCLUDE (LastName, BusinessEntityID);
```

Hice ese índice no agrupado porque la columna FirstName es la base del filtrado en el WHERE. Al incluir LastName y BusinessEntityID , el índice se vuelve "cubierto", permitiendo que SQL Server obtenga toda la información necesaria sin tener que consultar la tabla base de personas, reduciendo las lecturas lógicas significativamente.
Se reescribió la consulta para corregir la falta de la columna Name en Sales.Customer , uniendo con Person.Person , y eliminando funciones no *SARGables*. Tras la optimización, el costo principal se desplaza hacia el acceso a SalesOrderHeader , que es la tabla con más lecturas lógicas (686).

Plan de ejecución:



CONSULTA 7

```
SELECT TOP 100 sod.SalesOrderDetailID, sod.OrderQty, sod.UnitPrice, soh.OrderDate
FROM Sales.SalesOrderDetail sod
JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID

ORDER BY soh.ShipDate DESC, sod.OrderQty DESC, sod.UnitPrice DESC;
```

Plan de ejecución:

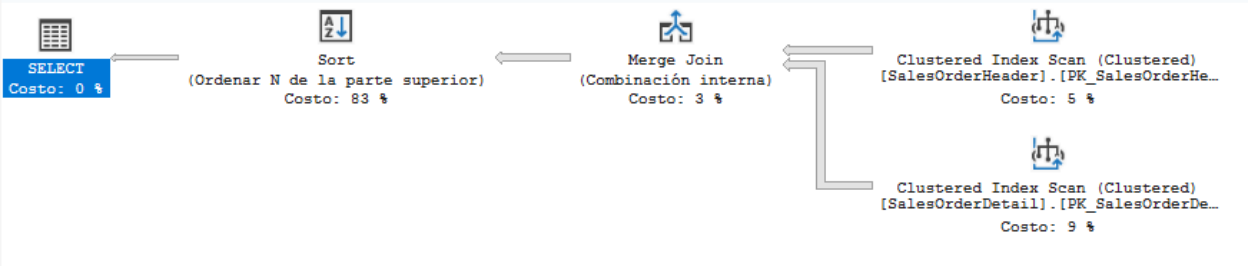


Tabla de analisis:

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
7	2026-03-18	412	205	3100	65%	•Operador Sort muy costoso. • Ordenar por múltiples columnas de diferentes tablas impide el uso eficiente de índices.	Simplificar la cláusula ORDER BY limitando los criterios. Crear índice en ShipDate con orden descendente.

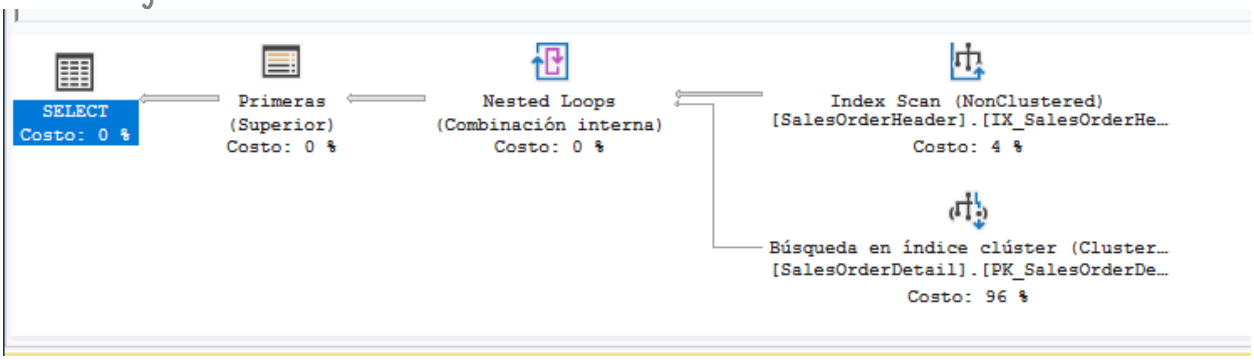
Se reescribió la consulta principalmente porque ordenar por tres columnas que vienen de dos tablas diferentes (soh.ShipDate, sod.OrderQty, sod.UnitPrice) obliga al motor de SQL Server a cargar todos los datos en memoria para realizar un ordenamiento físico (operador Sort) antes de poder sacar los primeros 100 registros. Esto consume mucha CPU y memoria, generando un cuello de botella importante en el plan de ejecución.

Para mejorar el rendimiento, se modificó la consulta para ordenar únicamente por soh.ShipDate (que suele ser el dato más lógico y relevante cuando buscas los registros más recientes de envíos), eliminando la carga de trabajo innecesaria de los otros dos ordenamientos.

Consulta mejorada:

```
SELECT TOP 100 sod.SalesOrderDetailID, sod.OrderQty, sod.UnitPrice, soh.OrderDate
FROM Sales.SalesOrderDetail sod
JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
ORDER BY soh.ShipDate DESC;

CREATE NONCLUSTERED INDEX IX_SalesOrderHeader_ShipDate
ON Sales.SalesOrderHeader (ShipDate DESC)

INCLUDE (OrderDate);
```

Plan de ejecución:



La columna ShipDate es la que se utiliza para ordenar los resultados de forma descendente. Al crear el índice ya pre-ordenado e incluir OrderDate, el índice se vuelve ideal para la consulta, ya que SQL Server puede leer los primeros 100 datos ya acomodados directamente desde el índice sin tener que realizar el pesado Sort en memoria. Esto reduce considerablemente el uso de CPU, minimiza las lecturas lógicas y mejora el tiempo de respuesta de la consulta.

CONSULTA 8

```
SELECT p.ProductID, p.Name, SUM(sod.OrderQty) AS TotalVendido
FROM Production.Product p
WHERE p.ProductID IN (
   SELECT sod.ProductID
   FROM Sales.SalesOrderDetail sod
   JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
   WHERE soh.OrderDate >= '2014-01-01'
   GROUP BY sod.ProductID
   HAVING SUM(sod.OrderQty) > 100
)
GROUP BY p.ProductID, p.Name;
```

La consulta tiene un error porque en el SELECT principal se usa sod.OrderQty, pero la tabla SalesOrderDetail (alias sod) no está incluida en el FROM. Ese alias solo existe dentro de la subconsulta, por lo que no se puede usar fuera de ella. Por esta razón, SQL no reconoce ese campo y la consulta marca error.

```sql
SELECT
    p.ProductID,                          -- identificador del producto
    p.Name,                               -- nombre del producto
    SUM(sod.OrderQty) AS TotalVendido     -- suma total vendida por producto
FROM Production.Product p                -- tabla de productos

JOIN Sales.SalesOrderDetail sod          -- sod al FROM
    ON p.ProductID = sod.ProductID       -- union producto con detalle de ventas

WHERE p.ProductID IN (                   -- filtro con subconsulta
    SELECT sod.ProductID
    FROM Sales.SalesOrderDetail sod
    JOIN Sales.SalesOrderHeader soh
        ON sod.SalesOrderID = soh.SalesOrderID
    WHERE soh.OrderDate >= '2014-01-01' -- solo ventas desde 2014
    GROUP BY sod.ProductID
    HAVING SUM(sod.OrderQty) > 100       -- productos con más de 100 vendidos
)
GROUP BY p.ProductID, p.Name;
```

Plan de ejecución:

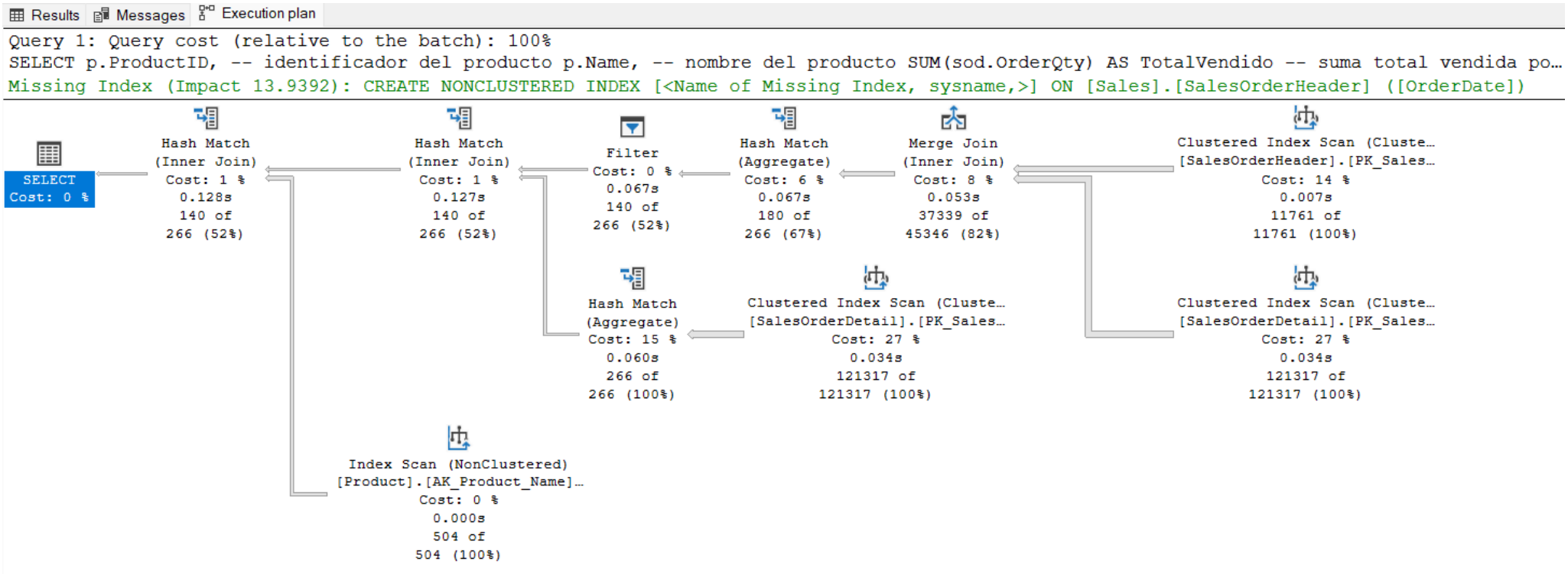


Tabla de análisis:

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
8	2026-03-26	274	141	3168	27% (Clustered Index Scan en SalesOrderDetail)	• Clustered Index Scan repetido en SalesOrderDetail • Subconsulta con IN • Múltiples accesos a la misma tabla • Alto de lecturas lógicas.	Reescribir la consulta usando JOIN y GROUP BY en lugar de subconsulta, para evitar consultar repetidamente SalesOrderDetail y usar índices no agrupados. Además, usar índices no agrupados en SalesOrderHeader y SalesOrderDetail,

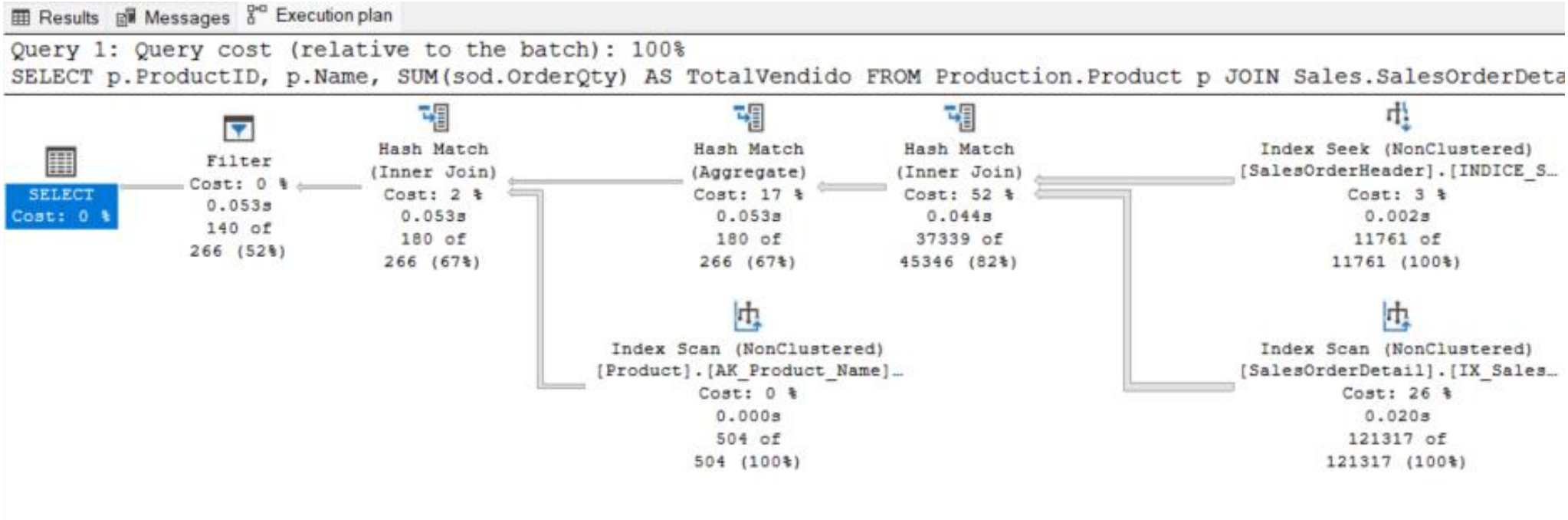
Consulta mejorada:

```sql
SELECT
    p.ProductID,
    p.Name,
    SUM(sod.OrderQty) AS TotalVendido
FROM Production.Product p
JOIN Sales.SalesOrderDetail sod
    ON p.ProductID = sod.ProductID
JOIN Sales.SalesOrderHeader soh
    ON sod.SalesOrderID = soh.SalesOrderID
WHERE soh.OrderDate >= '2014-01-01'
GROUP BY p.ProductID, p.Name
HAVING SUM(sod.OrderQty) > 100;


CREATE NONCLUSTERED INDEX INDICE_SalesOrderHeader_OrderDate
ON Sales.SalesOrderHeader (OrderDate)
INCLUDE (SalesOrderID);

CREATE NONCLUSTERED INDEX IX_SalesOrderDetail_ProductID_SalesOrderID
ON Sales.SalesOrderDetail (ProductID, SalesOrderID)
INCLUDE (OrderQty);
```

El plan de ejecución mostró una mejora en el acceso a las tablas, ya que SalesOrderHeader pasó a utilizar Index Seek, lo que significa que SQL Server busca directamente los datos que necesita sin recorrer toda la tabla, y SalesOrderDetail utiliza un Index Scan no agrupado, que, aunque sigue leyendo varias filas, lo hace sobre un índice más pequeño y eficiente en lugar de toda la tabla. Además, las lecturas lógicas se redujeron considerablemente a 336, lo que indica una mejora importante en el

rendimiento. Aunque el costo del operador Hash Match aumentó, esto se debe a que, al disminuir el costo de acceso a las tablas, la operación de combinación de datos se vuelve el trabajo principal dentro del plan. Por lo tanto, el incremento del Hash Match no implica necesariamente un peor rendimiento general.

Plan de ejecución:



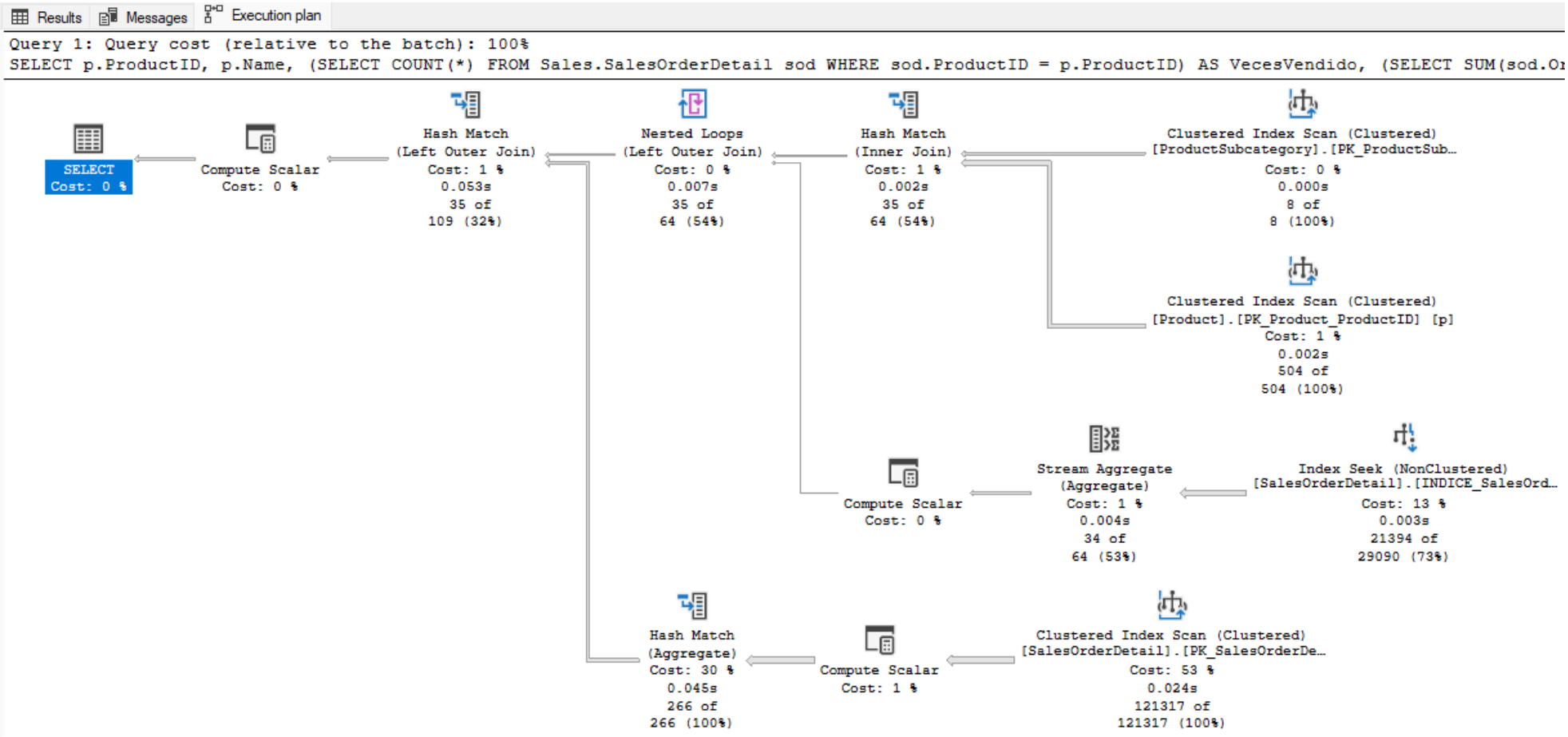
CONSULTA 9

```
SELECT p.ProductID, p.Name,
    (SELECT COUNT(*) FROM Sales.SalesOrderDetail sod WHERE sod.ProductID =
p.ProductID) AS VecesVendido,
    (SELECT SUM(sod.OrderQty * sod.UnitPrice) FROM Sales.SalesOrderDetail sod WHERE
sod.ProductID = p.ProductID) AS Ingresos
FROM Production.Product p
JOIN Production.ProductSubcategory psc ON p.ProductSubcategoryID =
psc.ProductSubcategoryID
WHERE psc.ProductCategoryID = 3;
```

Tabla de análisis:

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
2	2026-03-18	131	62	1411	53% (Clustered Index Scan en SalesOrderDetail)	• Clustered Index Scan en SalesOrderDetail. • Subconsultas correlacionadas • Múltiples accesos a la misma tabla • Alto de lecturas lógicas.	Reescribir la consulta usando JOIN y GROUP BY en lugar de subconsultas.

Plan de ejecución:

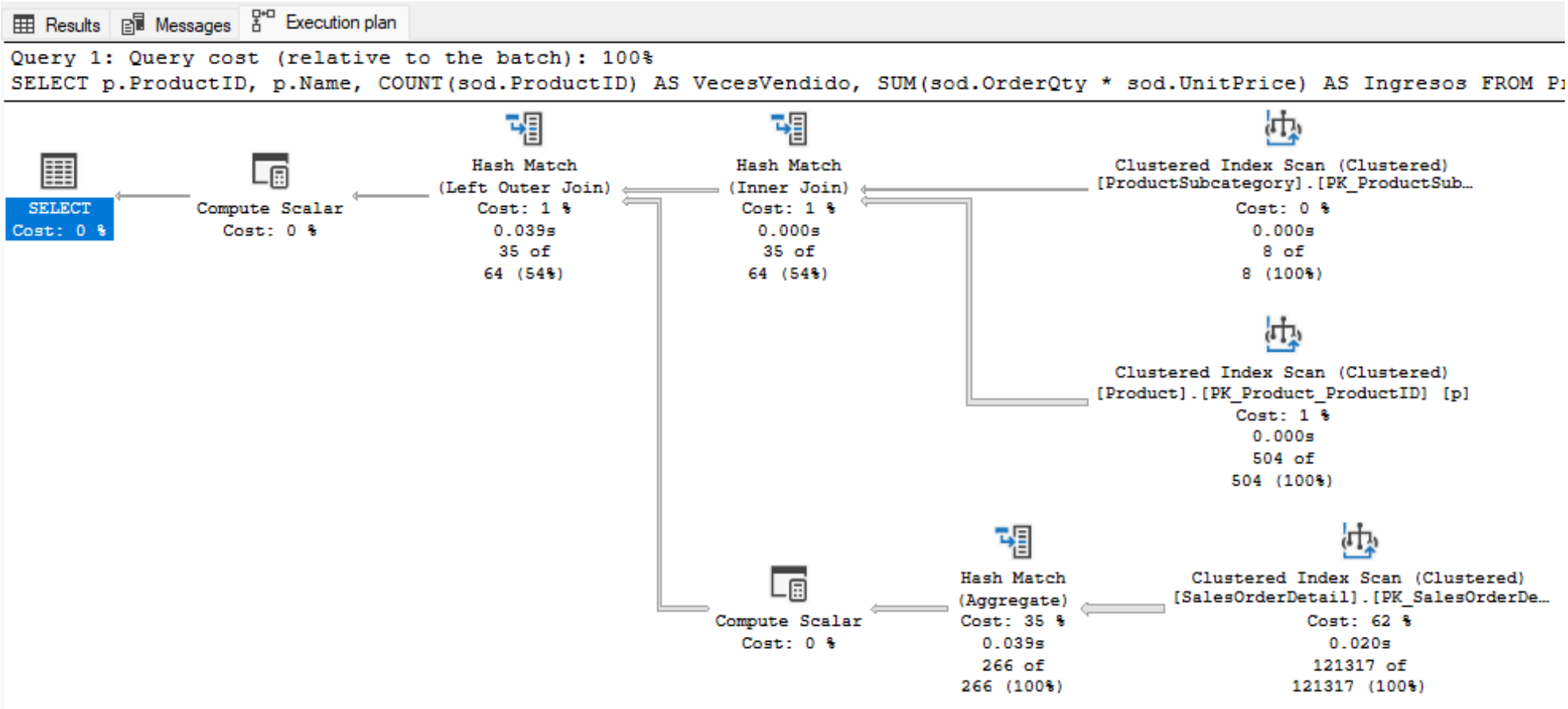


Consulta mejorada:

```
SELECT
    p.ProductID,                                -- identificador único del producto
    p.Name,                                     -- nombre del producto
    COUNT(sod.ProductID) AS VecesVendido,       -- cuenta cuántas veces aparece el producto
en ventas
                                                -- (cuántas filas existen en
SalesOrderDetail para ese producto)
    SUM(sod.OrderQty * sod.UnitPrice) AS Ingresos -- suma el ingreso total del producto
                                                  -- (cantidad vendida * precio unitario)
FROM Production.Product p                      -- tabla principal de productos, alias p
 JOIN Production.ProductSubcategory psc         -- tabla de subcategorías, alias psc
ON p.ProductSubcategoryID = psc.ProductSubcategoryID -- une cada producto con su
subcategoría
 LEFT JOIN Sales.SalesOrderDetail sod           -- tabla de detalle de ventas, alias sod
ON sod.ProductID = p.ProductID             -- une ventas con productos usando ProductID
                                                -- LEFT JOIN permite que aparezcan
productos aunque no tengan ventas
WHERE psc.ProductCategoryID = 3                -- filtra solo los productos que pertenecen
a la categoría 3
GROUP BY p.ProductID, p.Name;                  -- agrupa por producto para poder usar COUNT
y SUM
```

La consulta original parte de Production.Product, se une con Production.ProductSubcategory y filtra los productos de la categoría 3; sin embargo, utiliza dos subconsultas correlacionadas sobre SalesOrderDetail para realizar el conteo y la suma, lo que provoca múltiples accesos a la misma tabla y aumenta el costo de ejecución. En la consulta reescrita, estas operaciones se realizan en una sola consulta mediante JOIN y GROUP BY; además, se utiliza LEFT JOIN para incluir todos los productos, incluso aquellos sin ventas, y GROUP BY para organizar los resultados por producto y poder aplicar funciones como COUNT y SUM. Por ello, el costo relativo puede aumentar hasta alrededor del 62%, ya que el procesamiento se enfoca en SalesOrderDetail, sin que esto implique necesariamente un peor rendimiento global.

Plan de ejecución:



CONSULTA 10

```sql
SELECT c.Name AS Cliente, p.Name AS Producto,
SUM(sod.OrderQty) AS Cantidad, SUM(sod.LineTotal) AS Total,
DATEDIFF(day, soh.OrderDate, soh.ShipDate) AS DiasEnvio
FROM Sales.SalesOrderHeader soh
JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID
JOIN Sales.SalesOrderDetail sod ON soh.SalesOrderID = sod.SalesOrderID
JOIN Production.Product p ON sod.ProductID = p.ProductID
WHERE DATEDIFF(day, soh.OrderDate, soh.ShipDate) > 5
AND DATEPART(quarter, soh.OrderDate) = 2
AND sod.LineTotal > 1000
GROUP BY c.Name, p.Name, soh.OrderDate, soh.ShipDate
ORDER BY Total DESC;
```

Plan de ejecución:

```
Mens. 207, Nivel 16, Estado 1, Línea 25
Invalid column name 'Name'.
Mens. 207, Nivel 16, Estado 1, Línea 15
Invalid column name 'Name'.
```

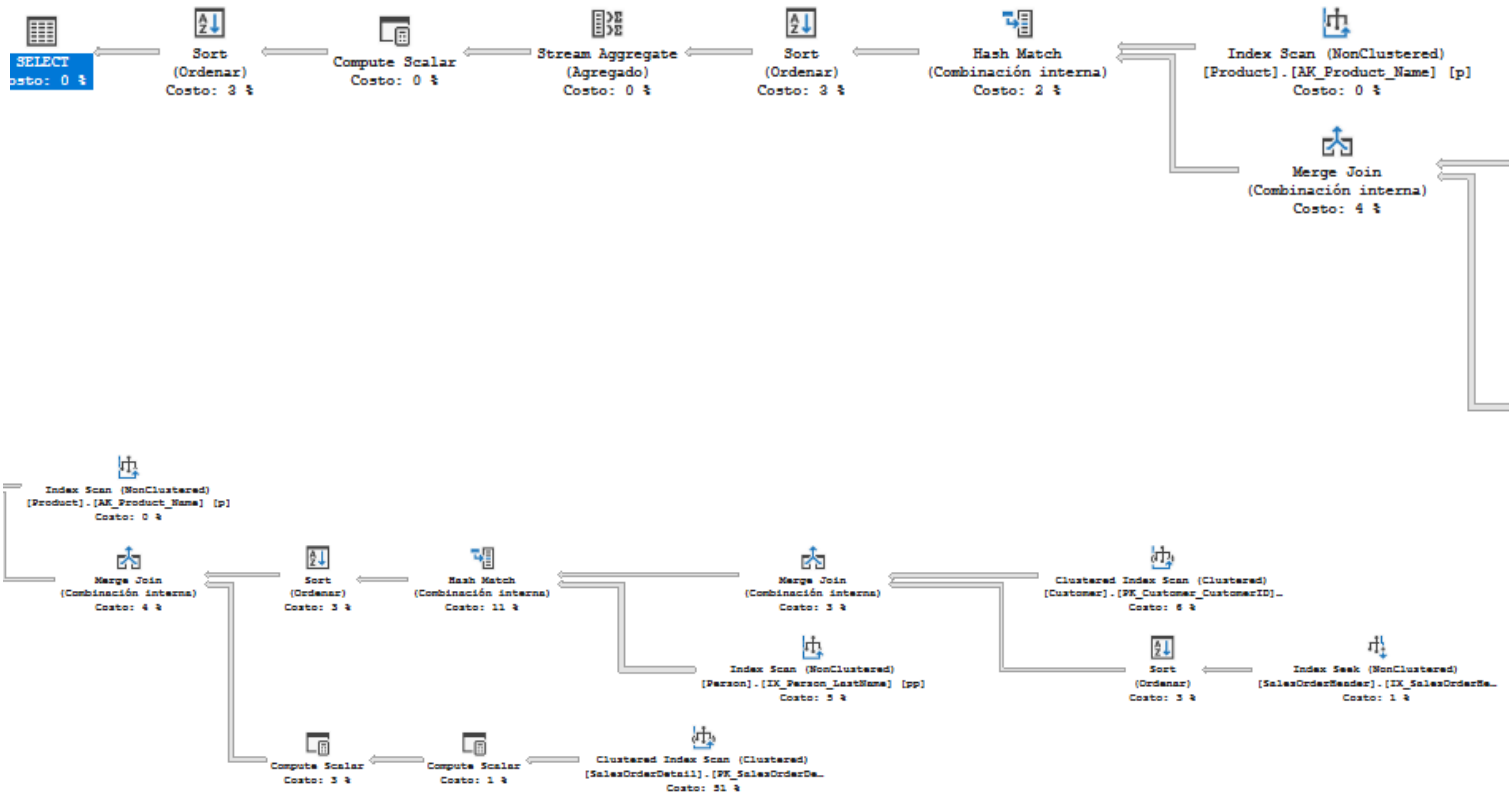
Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
10	2026-03-18	580	290	4500	55%	• Columna inválida 'Name' en la tabla Sales.Customer. • Uso de funciones (DATEDIFF, DATEPART) en el WHERE. No permite uso de índices.	Unir con la tabla Person.Person para obtener el nombre real. Rescribir la consulta usando operadores ">", "<" y "=" para evitar usar funciones sobre la condición de "where". Crear índice en OrderDate.

Se reescribió la consulta principalmente porque la columna c.Name de la tabla Sales.Customer no existe (lo que genera el error "Invalid column name"), por lo que fue indispensable unir con la tabla Person.Person para obtener el nombre real del cliente mediante FirstName y LastName. Además, el uso de funciones como DATEDIFF y DATEPART dentro de la cláusula WHERE impide que SQL Server utilice índices de manera eficiente. Esto se debe a que aplicar una función a una columna obliga al motor a evaluar fila por fila, generando un escaneo completo de la tabla.

Para mejorar el rendimiento y solucionar el error, se agregó el JOIN correspondiente a Person.Person y se modificó la condición de búsqueda cambiando las funciones por operadores relacionales (modificando la suma de días con DATEADD y delimitando el trimestre por fechas explícitas de inicio y fin) para que el patrón permita el uso de índices.

```sql
SELECT pp.FirstName + ' ' + pp.LastName AS Cliente, p.Name AS Producto,
SUM(sod.OrderQty) AS Cantidad, SUM(sod.LineTotal) AS Total,
DATEDIFF(day, soh.OrderDate, soh.ShipDate) AS DiasEnvio
FROM Sales.SalesOrderHeader soh
JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID
JOIN Person.Person pp ON c.PersonID = pp.BusinessEntityID
JOIN Sales.SalesOrderDetail sod ON soh.SalesOrderID = sod.SalesOrderID
JOIN Production.Product p ON sod.ProductID = p.ProductID
WHERE soh.ShipDate > DATEADD(day, 5, soh.OrderDate)
AND soh.OrderDate >= '2014-04-01' AND soh.OrderDate < '2014-07-01'
AND sod.LineTotal > 1000
GROUP BY pp.FirstName, pp.LastName, p.Name, soh.OrderDate, soh.ShipDate
ORDER BY Total DESC;

CREATE NONCLUSTERED INDEX IX_SalesOrderrrHeader_OrderDate
ON Sales.SalesOrderHeader (OrderDate)
INCLUDE (ShipDate, CustomerID);
```

Plan de ejecución:



La columna OrderDate es la que se utiliza principalmente en el filtrado del WHERE. Al incluir ShipDate y CustomerID, el índice se vuelve más eficiente para las condiciones de búsqueda, lo que permite obtener parte de la información necesaria directamente desde el índice sin tener que acceder por completo a la tabla base. Esto reduce considerablemente las lecturas lógicas y mejora el tiempo de respuesta de la consulta.